

1. Use o operador de unificação de Prolog para determinar se os seguintes termos unificam. Explique os casos em que os termos não unificam.

- (a) $f(X, a)$ e $f(f(Y, b), Y)$
- (b) $f(suc(z), X, z)$ e $f(X, Y, X)$
- (c) $g(X)$ e $f(Y)$
- (d) $g(f(X), X)$ e $g(Y, f(h(a)))$
- (e) $f(X)$ e X

2. Modele cada uma das palavras como um functor denominado `palavra` de aridade cinco. Exemplo: `palavra(c,a,r,t,a)`.

- carta
- canil
- cravo
- trama
- prato
- artes
- ratos

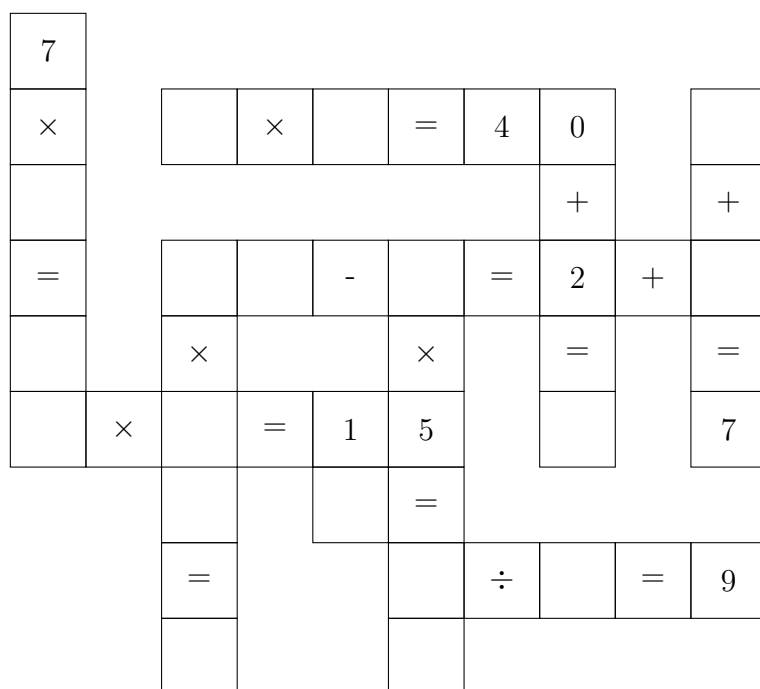
- (a) Construa um predicado denominado `testaPalavra` que, dados cinco argumentos, determine se existe alguma palavra correspondente aos argumentos fornecidos.
 - (b) Construa um predicado denominado `fimComum` tal que, dados 10 argumentos (duas palavras), determine se elas terminam com a mesma letra.
 - (c) Use os predicados definidos e construa uma consulta cujas soluções sejam todas as palavras que terminam com a mesma letra.
3. Construa o predicado `same(X,Y)` que aceite dois argumentos e determine se eles são iguais:
- (a) Por unificação.
 - (b) Por valor (ambos os termos devem estar instanciados com o mesmo valor).
 - (c) Mostre um exemplo em que dois termos são iguais por unificação, mas não por valor.
4. Assuma a seguinte definição de números naturais em Prolog.

```
nat(z).  
nat(s(X)) :- nat(X).  
  
soma(z, Y, Y).  
soma(s(X), Y, s(Z)) :- soma(X, Y, Z).
```

- (a) Defina a operação de multiplicação `mult(X,Y,Z)`.
 - (b) Defina a operação de divisão `div(X,Y,Z)`.
 - (c) Defina o predicado `isPrime(N)`.
 - (d) Defina o predicado `natToInt(N,I)`, que converte um natural em inteiro.
 - (e) Defina o predicado `intToNat(I,N)`, que converte um inteiro em natural.
5. Defina o predicado `perfectSquare(N)` tal que, dado um número inteiro e positivo `N`, resulte em sucesso se, e somente se, esse número for um quadrado perfeito.
6. Defina o predicado `repeat(N,P)` tal que, dado um número inteiro positivo e um predicado `P`, repita o predicado `N` vezes ou até que falhe.

7. Um número natural é dito ser um número de Mersenne¹ se ele tem a forma $2^p - 1$. Defina o predicado `mersenne(N,P)`, que, dado um número `N`, tenta determinar se existe algum valor $1 \leq p \leq N - 1$ tal que $N = 2^p - 1$.
8. Considere que um par de valores seja representado pelo functor `par` de aridade dois. Considere agora uma lista que contenha elementos definidos com o functor `par`. Defina o predicado `lookup(XS,X,Y)`, que resulta em sucesso se, e somente se, existe algum `Y` tal que o par `par(X,Y)`, esteja na lista `XS`.
9. Defina o predicado `mirror(XS,ZS)` que, dada uma lista `XS`, retorna um palíndromo formado a partir de `XS`. Exemplo: `mirror([m,i,r,r,o,r],ZS)` resulta em `ZS = [r,o,r,r,i,m,m,i,r,r,o,r]`.
10. Defina o predicado `statOf(XS,E,N)` que, dada uma lista `XS` e um elemento `E` retorna, na variável `N`, o número de ocorrências de `E` na lista.
11. Construa um programa em Prolog que aceite dois números representados por listas de dígitos e realize a soma desses dois números em uma terceira lista. Ex.: `somaDigits([1,2,3],[4,5,6],ZS)` deve resultar em `ZS = [5,7,9]`. Seu programa não pode converter a lista de dígitos em números.
12. Dizemos que l' é uma sublista de l se todos os elementos de l' ocorrem em l **na mesma ordem**. Defina o predicado `sublist(XS,YS)` que resulta em sucesso se, e somente se, `XS` é uma sublista de `YS`.
13. Construa um predicado que compute uma lista com n valores começando de 1 até n . Seu predicado deve ter apenas um parâmetro, no qual a lista é retornada. A princípio ele deverá retornar apenas a lista `[1]`. Ao se solicitar uma alternativa ao Prolog, o predicado deve produzir uma lista um elemento mais longa que a anterior.
14. Seja o predicado `wr3(L1,L2,L3)` que representa palavras de três letras. Construa o predicado `coincide(W1,W2)`, em Prolog, que, dadas duas palavras, resulte em sucesso se, e somente se, elas terminam com a mesma letra.
15. Um jogo de álgebra é comumente empregado para treinar crianças a realizarem as quatro operações básicas. O jogo é composto de um tabuleiro onde se dispõem simples expressões algébricas. O objetivo é preencher as células vazias com valores numéricos de modo que as expressões obtidas em linhas e colunas sejam corretas. A seguir, reproduz-se um desses jogos.

¹Números de Mersenne recebem esse nome em homenagem ao monge francês Marin Mersenne, que os estudou no século XVI.

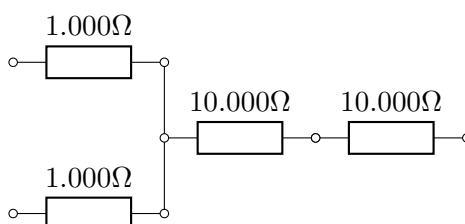


Modele esse jogo em Prolog e obtenha todas as soluções possíveis. Note que, em alguns casos, apenas um algarismo do resultado de uma expressão é usado como valor de outra expressão.

16. Construa um predicado tal que, dada uma árvore binária de busca, determine se ela está ou não balanceada.
17. Assuma que as operações lógicas $\wedge$, $\vee$ e $\neg$ sejam representadas pelos funtores `land/2`, `lor/2` e `lnot/1`, e que as constantes `true` e `false` sejam representadas pelos átomos `t` e `f`. Defina o predicado `eval(E,V)` que, dada uma fórmula da lógica proposicional `E`, determina o valor-verdade `V` dessa fórmula.
18. Uma rede de resistores pode ser representada em Prolog por meio dos seguintes termos:
 - `seq(R1,R2)`, que representa os resistores `R1` e `R2` em série.
 - `par(R1,R2)`, que representa os resistores `R1` e `R2` em paralelo.
 - `r(V)`, que representa um resistor com valor `V`.

Os valores dos resistores podem ser declarados como fatos. Suponha apenas 3 valores: 10.000Ω , 5.000Ω e 1.000Ω .

`?- res(N,20500).`
`N = seq(seq(par(r(1000), r(1000)), r(10000)), r(10000)).`



O circuito acima representa a resposta dada pela consulta.

Defina o predicado `res(N,V)` que compute o valor da resistência `V` da rede de resistores `N`. Seu predicado também deve funcionar de forma inversa, como exemplificado.

19. Defina o predicado `factors(N,K,XS)` que tome como parâmetro um número inteiro positivo `N` e um inteiro positivo `K`, e retorne a lista de fatores a partir de `K` tal que o produto dos elementos em `XS` resulte em `N`.

```
?- factors(811136,2,XS).
XS = [2, 2, 2, 2, 2, 2, 2, 6337] ;
false.
```

20. Defina a regra `prefix(XS,YS)` tal que `XS` é um prefixo de `YS`.

```
Ex.: prefix(XS,[a,b,c,d])
XS = []; XS = [a]; XS = [a,b].
```

21. A raiz digital de um número natural n é definida da seguinte forma:

1. Se $0 \leq n \leq 9$, então a raiz digital é o próprio n .
2. Caso contrário sejam $d_0, \dots, d_k$ os dígitos de n , então compute $\sum_{i=0}^k d_i$ e repita o passo 1 sobre o resultado.

Dizemos que um número n é auto-digital se sua raiz digital é um dos dígitos de n . Defina um predicado em Prolog que determina se um número inteiro positivo, incluindo zero, é auto-digital.

22. Defina uma regra que calcule o maior elemento de uma lista. Use o operador de corte de modo que o maior elemento retornado seja único.
23. Explique como funciona a negação por falha finita em Prolog. Mostre uma situação em que o **not** de Prolog não funciona como esperado.
24. Considere o seguinte programa em Prolog:

```
f(X,0): - X < 3.
f(X,1): - 3 <= X, X < 6.
f(X,2): - 6 <= X.
```

Insira operadores de corte de modo que **nenhuma solução produzida pelo programa atual seja rejeitada**, mas caminhos que levam a testes desnecessários não sejam explorados pela máquina de resolução de Prolog.

25. Mostre que Prolog é tão poderoso quanto a máquina de Turing. Faça isso mostrando como se pode simular uma máquina de Turing em Prolog.